

nginx spa in config, for fallback

added `try_files $uri $uri/ /index.html;` to `default.conf` of nginx to ensure that when the website doesn't say error 404 page not found

edges add pos from renderjson invert y for nodes and edges, as svg uses inverted direction (than vis.js) and then remake the centering CENTER ZOOM reactively window size (developer menu), center again dropdown -> normal manual chunking vite

In the course "Algorithmen und Datenstrukturen 2" (Algorithms and Data Structures 2) at the Johannes Kepler University, students learn about graphs and graph flow algorithms. Currently, the teaching materials for these topics are limited to slides and blackboard explanations. To enhance the learning experience for students and provide lecturers with more effective teaching aids, an interactive and playful visualization tool for graph and graph flow algorithms can be developed.

The goal of this bachelor's thesis is to create an interactive visualization tool that supports lecturers in teaching and helps students learn graph and graph flow algorithms more effectively. The tool should include the following features:

1. Graph building and editing: - Allow users to add, remove, and edit nodes and edges in a graph. - Enable users to change node names and edge weights to create custom graph structures. - Provide functionality to lay out graphs automatically for better visualization.

2. Visualization of various graph algorithms: - Implement visualizations for Depth-First Search (DFS), Breadth-First Search (BFS), and Dijkstra's shortest path algorithm. - Visualize the step-by-step process of each algorithm, highlighting the nodes and edges being traversed or updated. - As a nice-to-have feature, consider including the Ford-Fulkerson flow algorithm visualization. If this proves to be too complicated, focus on the other graph algorithms, especially Dijkstra's algorithm.

3. Modes for teaching and learning: - Step-by-step mode: Allow lecturers to present each algorithm's process step by step, with the ability to move forward and backward through the steps. - Play mode: Engage students by asking them questions about the algorithm's next steps. For example, in Dijkstra's algorithm, ask which node will be selected next and what information will be gained by this step. - Provide immediate feedback on the correctness of student answers and offer explanations for incorrect responses.

4. Web application: - Implement the interactive visualization tool as a web application using modern web technologies such as HTML, CSS, and JavaScript. - Design an intuitive and user-friendly interface that is easy to understand, navigate, and use. - Ensure compatibility with modern web browsers and responsiveness across different devices.

The interactive visualization tool should be developed with a focus on usability and educational value. It should provide a visually appealing and engaging experience for both lecturers and students, making it easier to teach and learn graph and graph flow algorithms.

```

// full dot support (viz.js cluster) // selected nodes with algorithm (click on
start node, calculating trace, prev/next button) // text editor visualizer (nope)
// directed graph layout of website (tabs, routing) // teaching mode (how)
DONE gray out removed selected node button and connect selected nodes
(disabled) connect button (nodex to nodex), different colors window close pre-
vent reload or close side (to prevent losing information html intercept close)
d3-graphviz dotstring d3-zoom
directed / undirected graph -i two tabs (routing) no dot support at all
(maybe later)
build directed tabs build undirected graph -i return dotstring load dotstring
and show graph -i Button BFS (undirected graph) -i DFS (undirected graph)
-i Dijkstra (undirected graph)
Build Undirected / Load from String Both start Dijkstra button (teaching
and student) teaching next, next, next slider and two buttons (prev and next)
that are synced student playing mode, guess correct mode
trace: TraceEntry[] = [] TraceEntry kind: (e.g. events enter, exit...) Pos-
sible trace for DFS: enter, x enter, y exit enter, z exit exit
teacher mode: curEntry = 0 stepNext(): entry = trace[curEntry]; exe-
cute(entry); curEntry++
stepPrev(): entry = trace[curEntry]; undo(entry) curEntry--
build_dijkstra_trace(start_node)...trace.push(kind: choose_edge, payload: [a, e]).....final_trace =
[chooseedge_a -> e, chooseedge_e -> f, chooseedge_e -> g]

```